Technical Document

1. Repository Directory Structure

This section defines the complete repository structure for the Customer Support AI system. The structure is intentionally designed to mirror the end-to-end decision pipeline described in Section 4 of Proposed_Solution.md.

The repository is not just organized for readability — it is structured to reflect how data flows through the system: Email → Interpretation → Extraction → Inference → Decision → Communication → Agent Assist → Escalation

Each directory corresponds to a stage in this pipeline or a supporting layer (infrastructure, shared logic, or documentation).

1.1 Top-Level Repository Structure

```
customer-support-ai/
│
├── apps/
│   ├── backend/
│   └── frontend/
│
├── supabase/
│
├── packages/
│   ├── prompts/
│   └── types/
│
├── docs/
│
├── scripts/
│
├── docker-compose.yml
├── .env.example
├── README.md
└── AGENTS.md
```

1.1.1 Conceptual Breakdown

The repository is divided into four core domains:

1. Execution Layer (apps/) Contains all runtime systems:
    - Backend: orchestration + decision engine
    - Frontend: agent-facing dashboard

2. Infrastructure Layer (supabase/) Defines:
    - database schema
    - storage (attachments)
    - edge functions

    This acts as the persistent memory of the system.

3. Shared Intelligence Layer (packages/) Contains:
    - prompts (LLM reasoning contracts)
    - shared types (frontend + backend)

    This ensures consistency in how the system interprets and reasons.

1.2 Backend System (apps/backend)

The backend is the core decision engine of the system. It is responsible for implementing every major capability defined in Section 4 of Proposed_Solution.md:

- 4.1 Multi-dimensional interpretation
- 4.2 Contextual acknowledgment
- 4.3 Missing information inference
- 4.4 Interaction depth control
- 4.5 Agent action planning
- 4.6 Dynamic urgency updates
- 4.7 Conditional escalation

1.2.1 Directory Structure

```
apps/backend/
│
├── app/
│   ├── main.py
│   ├── config.py
│
│   ├── api/
│   │   ├── routes/
│   │   │   ├── ingest.py
│   │   │   ├── tickets.py
│   │   │   ├── agent.py
│   │   │   └── webhook.py
│   │   └── deps.py
│
│   ├── services/
│   ├── repositories/
│   ├── models/
│   ├── llm/
│   ├── rag/
```

```
│   ├── workers/
│   └── utils/
│
├── tests/
├── requirements.txt
├── Dockerfile
└── .env
```

1.2.2 Core Application Layer (app/)

This directory contains the full execution logic of the backend system. It is structured to reflect the pipeline stages, ensuring that each transformation of data is isolated and traceable.

---

main.py

Acts as the entry point of the backend system.

Responsibilities:

- Initializes FastAPI
- Registers API routes
- Wires together services, repositories, and clients

It serves as the composition root where all components become a running system.

---

config.py

Centralized configuration module.

Responsibilities:

- Loads environment variables
- Stores API keys (Gemini, Supabase, Gmail)
- Defines system thresholds (confidence limits, interaction limits)

Purpose:

- Avoid configuration scattered across files
- Enable environment-specific behavior

1.2.3 API Layer (app/api/)

Defines how external systems interact with the backend.

This layer:

- validates inputs

- calls service layer
- returns structured responses

It does not contain business logic.

---

routes/ingest.py

Handles ingestion of incoming emails.

Responsibilities:

- Accept raw email payload
- Normalize into internal format
- Trigger processing pipeline

Maps to:

- Section 4.9 (pipeline initiation)

---

routes/tickets.py

Used by frontend to fetch ticket data.

Responsibilities:

- Provide ticket list
- Provide detailed ticket view

Ensures consistency across:

- interpretation
- extracted entities
- decisions

Maps to:

- Section 4.8 (consistency)

---

routes/agent.py

Handles agent-triggered actions.

Responsibilities:

- send replies
- override extracted data
- trigger escalation

Ensures:

- system supports human override

Maps to:

- Sections 4.5 and 4.7

---

routes/webhook.py

Handles inbound email provider callbacks.

Responsibilities:

- receive external email events
- forward to ingestion pipeline

---

deps.py

Defines dependency injection.

Responsibilities:

- initialize shared services
- manage database connections
- provide reusable dependencies

1.2.4 Services Layer (app/services/)

This is the core logic layer where the system's intelligence resides.

Each service corresponds to a stage in the pipeline.

---

ingestion_service.py

Transforms raw email input into structured ticket data.

Responsibilities:

- normalize email
- store ticket in database

Maps to:

- Section 4.9

---

interpretation_service.py

Performs multi-dimensional analysis of the email.

Outputs:

- intent
- category
- sentiment
- urgency
- confidence

This is the foundation of all downstream logic.

Maps to:

- Section 4.1

---

entity_extraction_service.py

Extracts structured data from:

- email body
- attachments via RAG

Examples:

- order_id
- invoice_id

Maps to:

- Section 4.3

---

requirement_inference_service.py

Determines required information dynamically using LLM.

Responsibilities:

- infer required fields
- avoid unnecessary requests

Maps to:

- Section 4.3

---

rag_service.py

Handles document intelligence.

Responsibilities:

- parse attachments

- generate embeddings
- retrieve relevant content

Enhances:

- entity extraction
- agent assist

Maps to:

- Section 4.5

---

decision_service.py

Determines system actions.

Inputs:

- interpretation
- extracted entities
- missing information
- interaction count

Outputs:

- ask for info or not
- escalate or not
- continue or stop automation

Maps to:

- Sections 4.4 and 4.7

---

communication_service.py

Generates customer-facing communication.

Responsibilities:

- acknowledgment messages
- structured information requests

Ensures clarity and tone control.

Maps to:

- Sections 4.2 and 4.3

---

agent_assist_service.py

Generates internal guidance for agents.

Outputs:

- summary
- verified facts
- action plan
- escalation suggestion

Maps to:

- Section 4.5

---

urgency_service.py

Updates urgency over time.

Purpose:

- prevent ticket starvation
- ensure fair prioritization

Maps to:

- Section 4.6

1.2.5 Repository Layer (app/repositories/)

Abstracts database access.

Responsibilities:

- handle all Supabase queries
- isolate SQL from business logic

1.2.6 Models Layer (app/models/)

Defines strict schemas.

Purpose:

- validate inputs and outputs
- ensure consistency across system layers

1.2.7 LLM Layer (app/llm/)

Handles all interactions with Gemini.

Responsibilities:

- execute prompts
- enforce structured outputs
- handle retries

1.2.8 RAG Layer (app/rag/)

Implements retrieval pipeline:

- parsing
- chunking
- embedding
- retrieval

This enables extraction of data from attachments.

1.2.9 Workers (app/workers/)

Handles asynchronous processing.

Examples:

- urgency updates
- document parsing

1.2.10 Utilities (app/utils/)

Contains:

- validation helpers
- logging utilities
- reusable helper functions

1.2.11 Tests (tests/)

Contains:

- unit tests
- integration tests
- pipeline tests

1.3 Frontend System (apps/frontend)

The frontend acts as the agent interaction layer.

Its purpose is to:

- provide visibility into system reasoning

- allow agents to act on tickets
- enable overrides where necessary

1.3.1 Structure

```
apps/frontend/
│
├── src/
│   ├── pages/
│   ├── components/
│   ├── services/
│   ├── hooks/
│   ├── store/
│   ├── types/
│   └── utils/
```

1.3.2 Functional Overview

The frontend enables agents to:

- view tickets and their interpretations
- inspect extracted data
- review missing information
- follow action plans
- override system decisions
- trigger escalation

1.4 Supabase (supabase/)

This directory defines the persistent infrastructure layer.

It includes:

- database schema (tables, relationships)
- storage configuration (attachments)
- edge functions (email handling, escalation)

This layer ensures that:

- all system state is persisted
- data can be retrieved and updated consistently
- integrations are handled at the infrastructure level

1.5 Shared Packages (packages/)

1.5.1 prompts/

Contains all LLM prompts used across the system.

Purpose:

- ensure prompt consistency
- enable version control of reasoning logic
- allow independent prompt iteration

1.5.2 types/

Contains shared schemas between frontend and backend.

Purpose:

- maintain consistency in data structures
- reduce duplication

1.6 Documentation (docs/)

This directory is critical for developer onboarding and long-term maintainability.

It contains:

- system-architecture.md → high-level design
- api-contracts.md → request/response formats
- rag-design.md → document processing logic
- decision-engine.md → decision-making rules
- escalation-rules.md → escalation criteria

1.8 Scripts (scripts/)

Contains utility scripts for development and testing.

Examples:

- simulate_email.py → inject test emails
- seed_db.py → populate database
- test_pipeline.py → run full pipeline

1.9 Root Files

README.md

Contains:

- project overview
- setup instructions
- architecture summary

.env.example

Template for required environment variables.

---

docker-compose.yml

Defines local development environment:

- backend
- frontend
- optional services (e.g., Redis)

Summary

This repository structure is intentionally aligned with the system pipeline:

Email → Interpretation → Extraction → Inference → Decision → Communication → Agent Assist → Escalation

Each directory corresponds to a stage in this flow or a supporting layer.

This ensures:

- clarity in responsibilities
- ease of debugging
- scalability of the system

2. Supabase Schema and Database Design

This section defines the complete database architecture for the system using Supabase (PostgreSQL). The schema is designed to act as the **persistent memory and state management layer** of the system.

Unlike traditional CRUD systems, this database is not just storing records — it is maintaining:

- evolving ticket state
- interpretation outputs
- extracted knowledge
- interaction history
- system decisions

Every table is aligned with one or more subsections of Section 4 of Proposed_Solution.md.

2.1 Design Philosophy

The schema is built around the following principles:

1. **Separation of Concerns** Each table represents a distinct concept:
    - tickets (core entity)
    - interpretations (LLM understanding)
    - entities (structured extracted data)
    - messages (conversation thread)
    - attachments (RAG layer)
    - agent_actions (decision outputs)
2. **Traceability** Every decision made by the system can be traced back to:
    - raw input
    - interpretation
    - extracted data
3. **Extensibility** The schema allows:
    - adding new entity types
    - evolving interpretation models
    - improving RAG without schema redesign
4. **Alignment with Pipeline** Each table maps to a pipeline stage:

Pipeline Stage	Table
Email ingestion	tickets
Interpretation	interpretations
Data extraction	entities
Communication	messages
Document intelligence	attachments
Agent guidance	agent_actions

2.2 Required Extensions

Before creating tables, the following extension must be enabled:

```
pgvector extension
```

Query: create extension if not exists vector;

Purpose:

- Enables vector similarity search
- Required for RAG (Section 4.5)

2.3 Core Tables

2.3.1 tickets

This is the central entity of the system.

Every incoming email becomes a ticket.

Maps to:

- Section 4.9 (system flow entry)
- Section 4.6 (urgency tracking)

Schema:

```
create table tickets (
    id uuid primary key default gen_random_uuid(),
    thread_id text,
    customer_email text not null,
    subject text,
    body text,
    status text default 'open',
    urgency_score float default 0.0,
    interaction_count int default 0,
    created_at timestamp default now(),
    updated_at timestamp default now()
);
```

Additional Queries

Indexing:

```
create index idx_tickets_created_at on tickets(created_at);
create index idx_tickets_status on tickets(status);
```

Trigger for updated_at:

```
create or replace function update_timestamp()
returns trigger as $$
begin
    new.updated_at = now();
    return new;
end;
$$ language plpgsql;
```

```
create trigger update_tickets_timestamp
before update on tickets
for each row execute function update_timestamp();
```

RLS (Row Level Security)

Enable:

```
alter table tickets enable row level security;
```

Policy (example — open access for backend service role):

```
create policy "Allow full access to service role"
on tickets
for all
using (true);
```

2.3.2 interpretations

Stores the LLM's understanding of the email.

Maps to:

- Section 4.1 (multi-dimensional interpretation)

Schema:

```
create table interpretations (
    id uuid primary key default gen_random_uuid(),
    ticket_id uuid references tickets(id) on delete cascade,
    intent text,
    category text,
    sentiment text,
    urgency float,
    confidence float,
    reasoning text,
    raw_output jsonb,
    created_at timestamp default now()
);
```

Indexes

```
create index idx_interpretations_ticket_id on interpretations(ticket_id);
```

RLS

```
alter table interpretations enable row level security;

create policy "Allow full access"
on interpretations
for all
using (true);
```

2.3.3 entities

Stores structured extracted data.

Maps to:

- Section 4.3 (information extraction)

Schema:

```
create table entities (
    id uuid primary key default gen_random_uuid(),
    ticket_id uuid references tickets(id) on delete cascade,
    key text,
    value text,
    source text,
    confidence float,
    created_at timestamp default now()
);
```

Indexes

```
create index idx_entities_ticket_id on entities(ticket_id);
create index idx_entities_key on entities(key);
```

RLS

```
alter table entities enable row level security;

create policy "Allow full access"
on entities
for all
using (true);
```

2.3.4 messages

Stores full conversation history.

Maps to:

- Section 4.2 (communication)
- Section 4.4 (interaction control)

Schema:

```
create table messages (
    id uuid primary key default gen_random_uuid(),
    ticket_id uuid references tickets(id) on delete cascade,
    sender text,
    content text,
    timestamp timestamp default now()
);
```

Indexes

```
create index idx_messages_ticket_id on messages(ticket_id);
```

RLS

```
alter table messages enable row level security;

create policy "Allow full access"
on messages
for all
using (true);
```

2.3.5 attachments

Stores file metadata and embeddings.

Maps to:

- Section 4.5 (RAG)

Schema:

```
create table attachments (
    id uuid primary key default gen_random_uuid(),
    ticket_id uuid references tickets(id) on delete cascade,
    file_url text,
    parsed_text text,
    embedding vector(1536),
    created_at timestamp default now()
);
```

Indexes

```
create index idx_attachments_ticket_id on attachments(ticket_id);
```

Vector index (for similarity search):

```
create index idx_attachments_embedding
on attachments
using ivfflat (embedding vector_cosine_ops)
with (lists = 100);
```

RLS

```
alter table attachments enable row level security;

create policy "Allow full access"
on attachments
for all
using (true);
```

2.3.6 agent_actions

Stores generated action plans.

Maps to:

- Section 4.5 (agent assistance)
- Section 4.7 (escalation)

Schema:

```
create table agent_actions (
    id uuid primary key default gen_random_uuid(),
    ticket_id uuid references tickets(id) on delete cascade,
    summary text,
    action_plan text,
    escalation_target text,
    generated_at timestamp default now()
);
```

Indexes

```
create index idx_agent_actions_ticket_id on agent_actions(ticket_id);
```

RLS

```
alter table agent_actions enable row level security;

create policy "Allow full access"
on agent_actions
for all
using (true);
```

2.4 Storage Configuration (Supabase Buckets)

A storage bucket must be created:

Bucket name: attachments

Purpose:

- store uploaded files (PDFs, images)
- referenced in attachments.file_url

Supabase Storage Policies for `attachments` Bucket

This section defines the required Row Level Security (RLS) policies for the Supabase storage bucket used to store customer attachments such as invoices, images, and documents.

The bucket must be configured as **private**. All access must be mediated through the backend using the service role key and signed URLs.

---

1. Enable RLS on Storage Objects

Run:

```
alter table storage.objects enable row level security;
```

---

2. Insert Policy (Upload via Backend Only)

Purpose: Allow backend services (using service role key) to upload files to the `attachments` bucket.

```
create policy "Allow backend upload to attachments"
on storage.objects
for insert
with check (
    bucket_id = 'attachments'
);
```

---

3. Select Policy (Deny Direct Public Access)

Purpose: Prevent direct file access from clients. All file access must happen via signed URLs generated by the backend.

```
create policy "Deny public read access to attachments"
on storage.objects
for select
using (
    false
);
```

---

4. Delete Policy (Backend Cleanup)

Purpose: Allow backend to delete files when tickets or attachments are removed.

```
create policy "Allow backend delete from attachments"
on storage.objects
for delete
using (
    bucket_id = 'attachments'
);
```

---

5. (Optional) Path-Based Restriction (Recommended for Future)

If files are stored using ticket-based paths:

Example path: attachments/{ticket_id}/file.pdf

You can enforce stricter control:

```
create policy "Scoped access by bucket only"
on storage.objects
for all
using (
    bucket_id = 'attachments'
);
```

---

6. Access Pattern (Important)

Files should never be accessed directly via public URLs.

Correct flow:

1. Backend uploads file to bucket
2. Backend stores file path in database
3. Frontend requests file access from backend
4. Backend generates signed URL:

   supabase.storage .from("attachments") .create_signed_url(path, expires_in)

5. Frontend uses signed URL to fetch file

---

7. Security Notes

- Do NOT make the bucket public
- Do NOT expose raw file paths to frontend
- Always use short-lived signed URLs (e.g., 60–300 seconds)

- Always perform uploads via backend only

---

8. Summary

This policy setup ensures:

- Secure storage of sensitive customer data
- Controlled access via backend
- Compatibility with RAG pipeline and attachment processing
- Prevention of accidental data exposure

2.5 Additional Considerations

2.5.1 Data Integrity

- All child tables use: on delete cascade
- Ensures: deleting a ticket removes all associated data

2.5.2 Schema Evolution

Future additions may include:

- audit_logs table
- escalation_logs table
- model_feedback table (for fine-tuning)

2.5.3 Performance Considerations

- Indexes added on:
  - ticket_id
  - created_at
  - vector embeddings
- Ensures:
  - fast retrieval
  - efficient RAG queries

Summary

This schema transforms Supabase from a simple database into:

- a state machine (tracking ticket lifecycle)
- a knowledge store (entities + attachments)
- a reasoning log (interpretations + agent actions)

Every table directly supports one or more capabilities defined in Section 4 of Proposed_Solution.md, ensuring that the system is not just functional, but structurally aligned with its intended behavior.

3. Environment Variables, Dependencies, and System Requirements

This section defines all **external dependencies, environment variables, and system-level prerequisites** required to run the application end-to-end.

Unlike a simple application, this system depends on multiple coordinated components:

- Supabase (database + storage)
- Gemini (LLM reasoning)
- Email infrastructure (Gmail API / SMTP)
- File parsing (RAG pipeline)
- Backend runtime (Python + FastAPI)
- Frontend runtime (React)

The goal of this section is to ensure that:

- A developer can set up the system without ambiguity
- All runtime dependencies are explicitly declared
- There are no hidden assumptions that break execution

3.1 Environment Variables Overview

All environment variables must be defined in:

- apps/backend/.env (backend)
- apps/frontend/.env (frontend)
- supabase/.env (optional, for local CLI usage)

A `.env.example` file must be maintained at root.

3.2 Backend Environment Variables

These variables are required for the backend system to function.

3.2.1 Supabase Configuration

```
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
SUPABASE_ANON_KEY
```

Purpose:

- SUPABASE_URL
  The base URL of your Supabase project

- SUPABASE_SERVICE_ROLE_KEY
  Used by backend for full access (bypasses RLS)
- SUPABASE_ANON_KEY
  Used optionally for client-safe operations

3.2.2 Gemini API Configuration

```
GEMINI_API_KEY
```

Purpose:

- Enables all LLM-powered services:
  - interpretation (4.1)
  - requirement inference (4.3)
  - agent assist (4.5)
  - communication generation (4.2)

3.2.3 Email (Gmail API / SMTP)

Option A: Gmail API

```
GMAIL_CLIENT_ID
GMAIL_CLIENT_SECRET
GMAIL_REFRESH_TOKEN
GMAIL_SENDER_EMAIL
```

Purpose:

- Read incoming emails
- Send replies

Option B: SMTP GMAIL_SMTP_HOST=smtp.gmail.com GMAIL_SMTP_PORT=587 GMAIL_SMTP_USER=your-email@gmail.com GMAIL_SMTP_PASSWORD=your-smtp-password

Purpose:

- Send outbound emails

For demonstration purposes, we will be using option B. Incoming emails will not be read - instead they will be fed in as user inputs in front-end.

3.2.4 RAG and File Processing

EMBEDDING_MODEL
EMBEDDING_DIMENSION

Example:

- EMBEDDING_MODEL = text-embedding-004 (Gemini)
- EMBEDDING_DIMENSION = 1536

Purpose:

- Required for vector storage and retrieval

3.2.5 Application Configuration

APP_ENV (development / production)
LOG_LEVEL (debug / info / warning)

INTERACTION_LIMIT (default: 2 or 3)
CONFIDENCE_THRESHOLD (default: 0.75)

Purpose:

- Control system behavior dynamically

3.2.6 Worker / Async Configuration (Optional but Recommended)

REDIS_URL

Purpose:

- Required if using Celery or async queues

3.3 Frontend Environment Variables

These variables are required for React frontend.

```
VITE_API_BASE_URL
VITE_SUPABASE_URL
VITE_SUPABASE_ANON_KEY
```

Purpose:

- Connect frontend to backend APIs
- Connect frontend to Supabase (optional direct queries)

3.4 Python Dependencies (Backend)

All dependencies must be listed in requirements.txt.

Core Framework

fastapi
uvicorn

Purpose:

- API server

Database & Supabase

supabase
psycopg2-binary

Purpose:

- Database interaction

LLM Integration

google-generativeai

Purpose:

- Gemini API interaction

RAG & File Processing

pymupdf
pdfplumber
pytesseract
pillow

Purpose:

- Parse PDFs and images

Embeddings

sentence-transformers (optional fallback)

Purpose:

- Generate embeddings if not using Gemini embeddings

Async / Workers

celery
redis

Purpose:

- Background task processing

Validation & Utilities

pydantic
python-dotenv
loguru

Purpose:

- Schema validation
- environment management
- logging

3.5 System-Level Dependencies

These must be installed on the machine (not via pip).

Tesseract OCR

Required for:

- extracting text from images

Installation:

Mac: brew install tesseract

Ubuntu: sudo apt install tesseract-ocr

PostgreSQL Extensions (Handled via Supabase)

- pgvector (already covered in Section 2)

3.6 Frontend Dependencies

Managed via package.json.

Core

react
react-dom
vite

State Management

zustand (recommended)

API Layer

axios

UI (Optional)

tailwindcss
shadcn/ui

3.7 Docker Dependencies (Optional but Recommended)

docker
docker-compose

Purpose:

- run backend + frontend together
- standardize environment

3.8 Minimum Working Configuration (MVP Checklist)

To run the system end-to-end, the following must be available:

- Supabase project with schema applied
- Storage bucket created
- Gemini API key configured
- Email sending configured (SMTP or Gmail)
- Backend dependencies installed
- Frontend dependencies installed
- Tesseract installed

3.9 Failure Modes if Misconfigured

This section highlights what breaks if dependencies are missing.

Missing Supabase Service Key:

- Backend cannot write data
- silent failures likely

Missing Gemini API Key:

- interpretation fails
- system cannot function

Missing Embeddings:

- RAG becomes non-functional
- attachments ignored

Missing Email Configuration:

- system cannot respond to users

Missing Tesseract:

- image attachments unusable

Missing Redis (if workers enabled):

- async jobs fail silently

Summary

This system depends on multiple external components working together.

The environment configuration ensures:

- consistent runtime behavior
- reproducibility across machines
- clarity for developers setting up the system

Once these variables and dependencies are correctly configured, the system is capable of running the full pipeline described in Section 4 of Proposed_Solution.md without requiring any training datasets.

4. Backend Implementation (End-to-End, File-by-File)

This section defines the **complete backend implementation**, starting from the lowest-level dependencies (configuration, DB client, LLM client) and moving upward to services and finally the FastAPI layer.

The structure follows **topological order**: A file is only introduced after all of its dependencies are already defined.

Every subsection includes:

- Purpose and role in the system
- Input and output schemas (strict JSON contracts where applicable)
- Internal flow
- Connection to Section 4 of Proposed_Solution.md
- How to run/test (smoke test + validation strategy)

This ensures that a developer can:

- implement incrementally
- test each layer independently
- verify correctness before moving forward

4.1 Configuration and Core Clients (Foundation Layer)

These files must be implemented first. All other components depend on them.

4.1.1 config.py

Purpose: Central configuration manager for the backend system.

Responsibilities:

- Load environment variables
- Provide strongly-typed access to configuration
- Define system constants

Expected Variables (loaded from .env):

```
SUPABASE_URL
SUPABASE_SERVICE_ROLE_KEY
GEMINI_API_KEY
SMTP_HOST / Gmail variables
EMBEDDING_MODEL
INTERACTION_LIMIT
CONFIDENCE_THRESHOLD
```

Internal Structure:

- A Config class or module-level variables
- Validation for missing variables
- Defaults for optional values

Output:

Python object accessible globally: config.SUPABASE_URL config.GEMINI_API_KEY

Smoke Test:

Run: python -c "from app.config import settings; print(settings.SUPABASE_URL)"

Expected:

- Correct value printed
- No runtime errors

4.1.2 supabase_client.py (inside repositories/ or utils/)

Purpose: Initialize and expose Supabase client.

Responsibilities:

- Connect to Supabase using service role key
- Provide reusable DB interface

Implementation:

- Use supabase-py client
- Initialize once (singleton pattern)

Output:

Function: get_supabase_client()

Returns: Supabase client instance

Smoke Test:

Run: python -c "from app.repositories.supabase_client import get_client; print(get_client())"

Expected:

- Valid client object
- No authentication errors

4.1.3 llm/client.py

Purpose: Wrapper for Gemini API.

Responsibilities:

- Send prompts
- Handle retries

- Return structured responses

Input:

```
{
    "prompt": "string"
}
```

Output:

```
{
    "text": "string",
    "raw": "full response"
}
```

Constraints:

- Must support deterministic JSON extraction
- Must retry on malformed output

Smoke Test:

Run a simple prompt: "Return JSON {"test": 1}"

Expected:

- Valid JSON response

4.1.4 llm/structured_output.py

Purpose: Ensure LLM responses are valid JSON.

Responsibilities:

- Validate JSON schema
- Retry or repair malformed output

Input:

```
{
    "raw_text": "string",
```

```
    "schema": {...}
}
```

Output:

```
{
    "validated_json": {...}
}
```

Smoke Test:

Pass intentionally malformed JSON → verify recovery

4.2 Data Access Layer (Repositories)

These files interact directly with Supabase.

4.2.1 ticket_repo.py

Purpose: CRUD operations for tickets

Functions:

create_ticket(input)

Input:

```
{
    "customer_email": "string",
    "subject": "string",
    "body": "string"
}
```

Output:

```
{
    "id": "uuid",
    "created_at": "timestamp"
}
```

---

get_ticket(ticket_id)

Output: Full ticket object

Smoke Test:

- Insert dummy ticket
- Retrieve it
- Verify values match

4.2.2 entity_repo.py

Purpose: Store and fetch extracted entities

Input:

```
{
    "ticket_id": "uuid",
    "key": "string",
    "value": "string",
    "confidence": 0.9
}
```

4.2.3 interpretation_repo.py

Purpose: Store LLM interpretation output

Input:

```
{
    "ticket_id": "uuid",
    "intent": "...",
    "category": "...",
    "confidence": 0.8
}
```

4.2.4 attachment_repo.py

Purpose: Store attachment metadata + embeddings

Input:

```
{
    "ticket_id": "uuid",
    "file_url": "string",
    "parsed_text": "string",
    "embedding": [vector]
}
```

4.3 RAG Layer

4.3.1 rag/parser.py

Purpose: Extract text from files

Input:

```
{
    "file_path": "string"
}
```

Output:

```
{
    "text": "extracted content"
}
```

Smoke Test:

- Upload PDF → verify text extraction

4.3.2 rag/indexer.py

Purpose: Store embeddings in DB

4.3.3 rag/retriever.py

Purpose: Fetch relevant chunks using vector similarity

Input:

```
{
    "query": "string",
    "ticket_id": "uuid"
}
```

Output:

```
{
    "chunks": ["..."]
}
```

4.4 Service Layer (Core Pipeline)

4.4.1 ingestion_service.py

Purpose: Start pipeline

Flow:

- receive email
- create ticket
- store message

Maps to: Section 4.9

4.4.2 interpretation_service.py

Purpose: Analyze email

Input:

```
{
    "ticket_id": "...",
    "body": "..."
}
```

Output:

```
{
    "intent": "...",
    "category": "...",
    "confidence": 0.8
}
```

Maps to: Section 4.1

4.4.3 entity_extraction_service.py

Purpose: Extract structured data

4.4.4 requirement_inference_service.py

Purpose: Infer missing fields dynamically

Input:

```
{
    "intent": "...",
    "entities": {...}
}
```

Output:

```
{
    "required_fields": ["invoice_id"]
}
```

Maps to: Section 4.3

4.4.5 decision_service.py

Purpose: Decide next action

Output:

```
{
    "ask_for_info": true,
    "escalate": false
}
```

Maps to: Sections 4.4, 4.7

4.4.6 communication_service.py

Purpose: Generate emails

Output:

```
{
    "email_body": "..."
}
```

Maps to: Section 4.2

4.4.7 agent_assist_service.py

Purpose: Generate agent plan

Output:

```
{
    "summary": "...",
    "steps": [...]
}
```

Maps to: Section 4.5

4.4.8 urgency_service.py

Purpose: Update urgency

Maps to: Section 4.6

4.5 API Layer (FastAPI)

4.5.1 ingest endpoint

POST /ingest

Input:

```
{
    "email": "...",
    "subject": "...",
    "attachments": []
}
```

Flow:

- ingestion_service
- interpretation
- extraction
- inference
- decision
- communication
- agent assist

Output:

```
{
    "ticket_id": "...",
    "status": "processed"
}
```

4.5.2 GET /tickets

Returns list of tickets

4.5.3 GET /tickets/{id}

Returns full ticket details

4.5.4 POST /agent/action

Input:

```
{
    "ticket_id": "...",
    "action": "escalate"
}
```

4.6 End-to-End Smoke Test

1. Call /ingest with test email
2. Verify:
    - ticket created
    - interpretation stored
    - entities extracted
    - missing fields inferred
    - response generated
3. Fetch ticket via API
4. Verify frontend consistency

Summary

This backend system is a fully connected pipeline where:

- Each service transforms state
- Each output feeds the next stage
- Every step is stored and traceable

This ensures:

- debuggability
- modularity
- alignment with the system design

5. Frontend Implementation (React Agent Dashboard)

This section defines the complete frontend implementation for the system. The frontend is not just a visualization layer — it is a **control interface for the decision system**.

It is responsible for:

- Exposing system reasoning transparently to agents
- Allowing human override of automated decisions
- Providing a structured interface to act on tickets

- Maintaining consistency with backend state

Every component in the frontend maps to one or more subsections of Section 4 of Proposed_Solution.md, especially:

- 4.2 (communication clarity)
- 4.3 (missing information visibility)
- 4.5 (agent action plan)
- 4.7 (escalation control)
- 4.8 (consistency across layers)

5.1 Architectural Overview

The frontend is a **React + Vite application** structured around:

- Pages (route-level views)
- Components (UI building blocks)
- Services (API communication)
- Hooks (state + data fetching logic)
- Store (global state management)

The frontend does NOT contain business logic. It is strictly:

- a consumer of backend APIs
- a renderer of system state
- a trigger for agent actions

5.2 Directory Structure

```
apps/frontend/
│
├── src/
│   ├── main.tsx
│   ├── App.tsx
│
│   ├── pages/
│   │   ├── Dashboard.tsx
│   │   └── TicketDetail.tsx
│
│   ├── components/
│   │   ├── TicketList.tsx
│   │   ├── TicketCard.tsx
│   │   ├── AgentPanel.tsx
│   │   ├── EntityViewer.tsx
│   │   ├── AttachmentViewer.tsx
│   │   └── ActionControls.tsx
│
│   ├── services/
│   │   ├── api.ts
```

```
│   │   └── supabaseClient.ts
│
│   ├── hooks/
│   │   ├── useTickets.ts
│   │   └── useTicketDetail.ts
│
│   ├── store/
│   │   └── ticketStore.ts
│
│   ├── types/
│   │   └── ticket.ts
│
│   └── utils/
│       └── format.ts
│
├── public/
├── package.json
└── vite.config.ts
```

5.3 Core Application Files

5.3.1 main.tsx

Purpose: Entry point of the React application.

Responsibilities:

- Mount React app to DOM
- Initialize global providers (store, router)

Execution:

- Automatically run by Vite

5.3.2 App.tsx

Purpose: Defines routing structure.

Responsibilities:

- Route definitions:
  - / → Dashboard
  - /ticket/:id → TicketDetail

5.4 Services Layer (Frontend API Communication)

5.4.1 services/api.ts

Purpose: Centralized API client for backend communication.

Responsibilities:

- Define base URL
- Handle all HTTP requests

Functions:

fetchTickets()

GET /tickets

Response: [{ "id": "uuid", "subject": "string", "status": "open", "urgency_score": 0.6 }]

---

fetchTicketById(ticket_id)

GET /tickets/{id}

Response: { "id": "uuid", "subject": "...", "body": "...", "interpretation": {...}, "entities": [...], "messages": [...], "agent_action": {...} }

---

sendAgentAction(payload)

POST /agent/action

Input: { "ticket_id": "uuid", "action": "reply | escalate | resolve", "data": {...} }

5.4.2 services/supabaseClient.ts

Purpose: Optional direct connection to Supabase.

Use cases:

- real-time updates
- listening to ticket changes

5.5 State Management

5.5.1 store/ticketStore.ts

Purpose: Global state store (using Zustand).

State:

```
{
    tickets: [],
```

```
    selectedTicket: null,
    loading: false
}
```

Actions:

- setTickets()
- setSelectedTicket()
- updateTicket()

5.6 Hooks Layer

5.6.1 useTickets.ts

Purpose: Fetch and manage ticket list.

Flow:

- call fetchTickets()
- update store

5.6.2 useTicketDetail.ts

Purpose: Fetch detailed ticket data.

Flow:

- call fetchTicketById()
- store result

5.7 Pages

5.7.1 Dashboard.tsx

Purpose: Display list of tickets.

Responsibilities:

- call useTickets()
- render TicketList

5.7.2 TicketDetail.tsx

Purpose: Display full ticket view.

Responsibilities:

- call useTicketDetail()
- render:
  - email content
  - interpretation
  - entities
  - attachments
  - agent panel

5.8 Components

5.8.1 TicketList.tsx

Purpose: Render list of tickets.

Input: [{ "id": "...", "subject": "...", "urgency_score": 0.7 }]

5.8.2 TicketCard.tsx

Purpose: Single ticket preview.

Displays:

- subject
- status
- urgency

5.8.3 AgentPanel.tsx

Purpose: Display agent assistance.

Input: { "summary": "...", "action_plan": [...], "escalation_target": "finance" }

Maps to:

- Section 4.5

5.8.4 EntityViewer.tsx

Purpose: Display extracted entities.

Input: [{"key": "order_id", "value": "12345"}]

Features:

- editable fields

- save changes

Maps to:

- Section 4.3

5.8.5 AttachmentViewer.tsx

Purpose: Display attachments.

Features:

- preview PDFs/images
- show extracted text (optional)

Maps to:

- Section 4.5

5.8.6 ActionControls.tsx

Purpose: Trigger agent actions.

Actions:

- reply
- escalate
- resolve

Calls: sendAgentAction()

Maps to:

- Section 4.7

5.9 Data Flow (Frontend ↔ Backend)

---

1. Dashboard loads → GET /tickets
2. User selects ticket → GET /tickets/{id}
3. TicketDetail renders:
    - interpretation
    - entities
    - agent plan
4. Agent performs action → POST /agent/action
5. Backend updates state → frontend refreshes

5.10 Local Execution

Run frontend:

```
npm install
npm run dev
```

Runs on: http://localhost:5173

5.11 Smoke Testing Frontend

Test 1: Dashboard Load

- Open app
- Verify tickets appear

Test 2: Ticket Detail

- Click ticket
- Verify:
  - interpretation visible
  - entities shown
  - agent plan displayed

Test 3: Action Trigger

- Click escalate
- Verify backend API called
- Verify UI updates

5.12 Failure Modes

API not reachable:

- no data displayed

Incorrect response shape:

- UI crashes

State mismatch:

- stale data shown

5.13 Frontend Wireframes (Agent Dashboard)

This section provides low-fidelity wireframes for the agent dashboard. The goal is not visual polish, but **clarity of layout, hierarchy, and interaction flow**.

Each wireframe maps directly to system capabilities described in Section 4 of Proposed_Solution.md.

---

1. Overall Layout (Primary Screen)

This is the main working interface used by agents.

```
+----------------------+----------------------------------------------+
|                      |                                              |
|   Ticket List        |   Ticket Detail + Agent Panel                |
|                      |                                              |
|  ------------------  |   ----------------------------------------   |
|  TicketCard         |   Email View (Thread)                      |
|  TicketCard         |   ----------------------------------------   |
|  TicketCard         |   Interpretation Panel                     |
|  TicketCard         |   ----------------------------------------   |
|                      |   Entities + Missing Info                  |
|                      |   ----------------------------------------   |
|                      |   Attachments                              |
|                      |   ----------------------------------------   |
|                      |   Agent Action Plan                        |
|                      |   ----------------------------------------   |
|                      |   Action Controls                          |
|                      |                                              |
+----------------------+----------------------------------------------+
```

---

2. Ticket List Panel (Left)

Purpose:

- Quickly scan tickets
- Prioritize based on urgency

  +----------+ | 🔍 Search / Filters | +----------+ | [HIGH] Order issue - refund | | user@email.com | | 2 hrs ago | +----------+ | [MED] Delivery delayed | | user@email.com | | 5 hrs ago | +----------+ | [LOW] Product query | | user@email.com | | 1 day ago | +----------+

Design Notes:

- Urgency color-coded:
  - HIGH → red

    - MED → yellow
    - LOW → gray
- Clicking a ticket loads detail view

Maps to:

- Section 4.6 (urgency prioritization)

---

3. Email View (Top Right)

Purpose:

- Show original issue and conversation

  +----------+ | Subject: Refund issue | | From: user@email.com | | | | "I was charged twice for my order..." |

  Previous Messages (if any)

  +----------+

Maps to:

- Section 4.2 (communication context)

---

4. Interpretation Panel

Purpose:

- Show system understanding

  +----------+

  🧠 **System Interpretation**

  Intent: Refund Request

  Category: Billing

  Sentiment: Frustrated

  Confidence: 0.82

  +----------+

Design Notes:

- Confidence < 0.6 → show warning
- Helps agent trust system output

Maps to:

- Section 4.1

---

5. Entities + Missing Info Panel

Purpose:

- Show extracted and missing data

  +----------+

  📦 **Extracted Data**

  Order ID: 12345

  Invoice ID: —

  Payment Method: Credit Card

  +----------+

  +----------+

  ⚠ **Missing Information**

  - Invoice ID

  +----------+

Design Notes:

- Missing fields highlighted in red
- Editable fields for agent correction

Maps to:

- Section 4.3

---

6. Attachment Viewer

Purpose:

- Show uploaded files and extracted content

  +----------+

  📎 **Attachments**

  invoice_123.pdf

  [Preview Button]

📎 **Attachments**

Extracted Content:

"Invoice ID: INV-98765"

+----------+

Design Notes:

- Toggle for "show extracted text"
- Supports RAG debugging

Maps to:

- Section 4.5

---

7. Agent Action Panel (Core Component)

Purpose:

- Provide clear next steps

+----------+

🤖 **Suggested Action Plan**

Summary:

Customer was charged twice for order.

Steps:

1. Verify order details
2. Check duplicate transaction
3. Initiate refund

Escalation: Not Required

+----------+

Design Notes:

- This is the most important block
- Should feel like “instruction mode”

Maps to:

- Section 4.5

---

8. Action Controls (Bottom Panel)

Purpose:

- Allow agent actions

```
+-----------------------------------------------+

  ✉ Reply
 ------------------------------------------------
  [Text Input Box]
 ================================================
 ------------------------------------------------
  [Send Reply]
 ------------------------------------------------
  +-----------------------------------------------+

+-----------------------------------------------+

  ⚙ Actions
 ------------------------------------------------
  [Escalate to Finance]
 ------------------------------------------------
  [Mark as Resolved]
 ------------------------------------------------
  +-----------------------------------------------+
```

Design Notes:

- Disable escalate if missing info exists
- Pre-fill reply suggestion (optional)

Maps to:

- Section 4.7

---

9. Interaction Flow

1. Agent opens dashboard
2. Sees prioritized ticket list
3. Clicks ticket
4. Reviews:
    - email
    - interpretation
    - extracted data
    - missing info
5. Follows action plan
6. Performs action (reply / escalate / resolve)

---

10. Key UX Principles

- Always show system reasoning
- Minimize agent cognitive load
- Make next action obvious
- Prevent invalid actions
- Keep layout consistent across tickets

Summary

These wireframes represent a **decision-support interface**, not just a dashboard.

The UI ensures:

- transparency of system reasoning
- clarity of required actions
- minimal back-and-forth
- tight integration with backend pipeline

5.14 Design Theme (Frontend Dashboard)

This section defines the **visual design system and UI constraints** for the agent dashboard. The objective is to maintain a **minimal, focused, and distraction-free interface** that complements the decision-support nature of the system.

The design must prioritize:

- clarity over aesthetics
- readability over decoration
- consistency over experimentation

---

1. Design Philosophy

The dashboard is not intended to be visually complex or expressive. It must:

- minimize cognitive load for agents
- avoid unnecessary visual noise
- emphasize structured information hierarchy
- ensure that actionable elements are immediately identifiable

The UI should feel:

- controlled
- calm
- predictable

---

2. Typography

Primary Font:

```
Fira Code
```

Usage:

- All text across the application must use Fira Code
- This includes:
  - headings
  - labels
  - body text
  - input fields
  - buttons

Reasoning:

- Monospaced font improves readability of structured data
- Aligns well with technical and data-heavy interfaces
- Reinforces consistency across extracted fields and system outputs

---

3. Color System

Primary Theme Color

```
#FFA749
```

This is the dominant accent color and should be used for:

- highlights
- active elements
- selected states
- primary buttons

Secondary Colors

Only the following palette is allowed:

- shades of #FFA749 (lighter and darker variants)
- grayscale tones:
  - dark gray (backgrounds)
  - medium gray (containers)
  - light gray (borders and separators)

Strict Restrictions

The following must NOT be used anywhere in the UI:

- blue (any shade)
- green (any shade)
- gradients of any kind
- multi-color highlights

Reason:

- maintain visual consistency
- avoid distraction
- reinforce a single visual identity

---

4. Background and Contrast

- Background should be dark gray or near-black
- All text must be:

  color: white
- Secondary text may use lighter gray (but remain readable)

This ensures:

- high contrast
- reduced eye strain
- clear hierarchy

---

5. Component Styling

Borders

- Circular or rounded borders must be used throughout
- No sharp edges

Recommended:

```
border-radius: 12px to 20px
```

Applies to:

- cards
- panels
- buttons
- input fields

Shadows

- Minimal or no shadows
- Prefer separation via spacing and contrast

Spacing

- Generous padding between sections
- Clear visual grouping of components

---

6. Button Design

Primary Button:

- Background: #FFA749
- Text: white
- Rounded corners

Secondary Button:

- Background: transparent or dark gray
- Border: light gray
- Text: white

Hover State:

- Slightly darker/lighter shade of #FFA749
- No animations or gradients

---

7. Panel Structure

Each panel (e.g., interpretation, entities, agent plan) should:

- be enclosed in a rounded container
- have consistent padding
- be separated by spacing, not lines

Hierarchy should be conveyed using:

- spacing
- font weight
- subtle color variation

---

8. Interaction Feedback

- Avoid flashy animations

- Use subtle transitions only (if necessary)
- Feedback should be immediate and minimal

---

9. Consistency Rules

- Same component must look identical across all pages
- Same data type must be styled consistently
- No experimental styles per component

---

10. Summary

The design system is intentionally restrictive.

It enforces:

- a single dominant color (#FFA749)
- grayscale support palette
- strict avoidance of blue, green, and gradients
- consistent use of Fira Code
- rounded, minimal UI elements
- white text for clarity

This ensures the dashboard remains:

- focused
- readable
- aligned with its role as a decision-support system

Summary

The frontend acts as:

- a visualization layer for system reasoning
- a control interface for agents
- a bridge between automation and human decision-making

It ensures that the system remains:

- transparent
- controllable
- consistent with backend state